3. Click ↻ **SCAN DONGLES** to verify device detection.

4. *(Optional)* Click ⚡ **AUTO-ASSIGN UNIQUE SERIALS** to burn permanent serial numbers (`00000001`, `00000002`...) into dongle EEPROMs.

5. In the **Stations** tab, select 📻 **Off-Air RTL-SDR**, enter your frequency (e.g. `105.1`), and click **ADD STATION**!

🛠️ Model Context Protocol (MCP) Tools

Radio SONAR includes a built-in MCP server for AI pair programmers:

MCP Tool
Description
`barracuda_sdr_status`
Retrieve status of all connected dongles, RF telemetry, and wideband clusters
`barracuda_sdr_tune`
Programmatically tune an SDR receiver (`freq`, `mode`, `program`, `gain`, `device`)
`barracuda_get_status`
Get active station monitors, now playing tracks, ad detection states, and RF meters
`barracuda_start_monitor`
Start monitoring an audio stream or off-air frequency
`barracuda_stop_monitor`
Stop an active monitor
`barracuda_list_stations`
List all saved stations and presets
`barracuda_get_recordings`
List captured commercial MP3 recordings and Whisper AI transcripts

📄 License

This project is licensed under the MIT License — see the LICENSE file for details.

```
cd radio-sonar

# Start the application
docker-compose up --build -d
```

Open your browser and navigate to `http://localhost:8090` (or your Unraid/server IP).

2. Docker Compose Configuration

```
services:
  recorder:
    build: ./recorder
    restart: unless-stopped
    volumes:
      - ./recorder/server.js:/app/server.js
      - ./recorder/audio_dsp.js:/app/audio_dsp.js
      - ./recorder/transcriber.js:/app/transcriber.js
      - ./recorder/sdr_receiver.js:/app/sdr_receiver.js
      - ./recorder/wideband_channelizer.py:/app/wideband_channelizer.py
      - ./recorder/shazam_recognize.py:/app/shazam_recognize.py
      - ./recorder/mcp_server.js:/app/mcp_server.js
      - recordings:/recordings
      - whisper_cache:/root/.cache/whisper
    devices:
      - /dev/bus/usb:/dev/bus/usb   # USB passthrough for RTL-SDR dongles
    networks:
      - internal

  web:
    build:
      context: .
      dockerfile: web.Dockerfile
    restart: unless-stopped
    ports:
      - "8090:80"
    volumes:
      - ./frontend/index.html:/usr/share/nginx/html/index.html:ro
      - recordings:/usr/share/nginx/html/recordings:ro
    networks:
      - internal

volumes:
  recordings:
  whisper_cache:

networks:
  internal:
```

NOTE

RTL-SDR USB Permissions: The container automatically blacklists default kernel DVB drivers (`dvb_usb_rtl28xxu`) so `libusb` can claim the receiver dongles with zero host configuration.

📡 RTL-SDR Hardware Setup

1. **Plug in your RTL-SDR dongle(s)** to your server or a powered USB hub.

2. In Radio SONAR, click 🔧 **Tools $\rightarrow$ 📻 RTL-SDR Receiver & Diagnostics**.

Batch Controls: 1-click `● REC ALL` and `🎙 AI ALL` controls for managing entire station fleets instantly.

🔌 Model Context Protocol (MCP) Server

Exposes full station control, monitoring, ad logs, and SDR hardware diagnostics to AI agent pair programmers via Model Context Protocol tools.

🏗️ Architecture

```
graph TD
    subgraph "Audio Sources"
        USB["RTL-SDR USB Dongles (/dev/bus/usb)"]
        WEB["Web Streams (HLS / ICY / Triton / TuneIn)"]
    end

    subgraph "Demodulation & SDR Layer"
        USB --> POOL["SdrPoolManager (recorder/sdr_receiver.js)"]
        POOL -->|"Analog Mode"| RTL_FM["rtl_fm + redsea (FM & RDS)"]
        POOL -->|"HD Mode"| NRSC5["nrsc5 (Digital HD1-HD4)"]
        POOL -->|"Wideband Mode"| WB["wideband_channelizer.py (2.4 MSps IQ)"]
    end

    subgraph "Core DSP & Ingestion"
        RTL_FM --> STREAM["/api/sdr/stream/:id.mp3"]
        NRSC5 --> STREAM
        WB --> STREAM
        WEB --> STREAM_WEB["Stream Ingester (audio_dsp.js)"]
    end

    subgraph "Intelligence & Detection Engine"
        STREAM --> DETECT["Ad Break & Music Detector"]
        STREAM_WEB --> DETECT
        DETECT --> FP["Acoustic Fingerprinter (Chromaprint / Shazam)"]
        DETECT --> REC["Commercial MP3 Audio Recorder"]
        REC --> WHISPER["Whisper AI Speech-to-Text"]
    end

    subgraph "User Interface & AI Tools"
        DETECT --> UI["Web UI (Port 8090)"]
        WHISPER --> UI
        UI --> MCP["Model Context Protocol (MCP) Server"]
    end
```

🚀 Getting Started

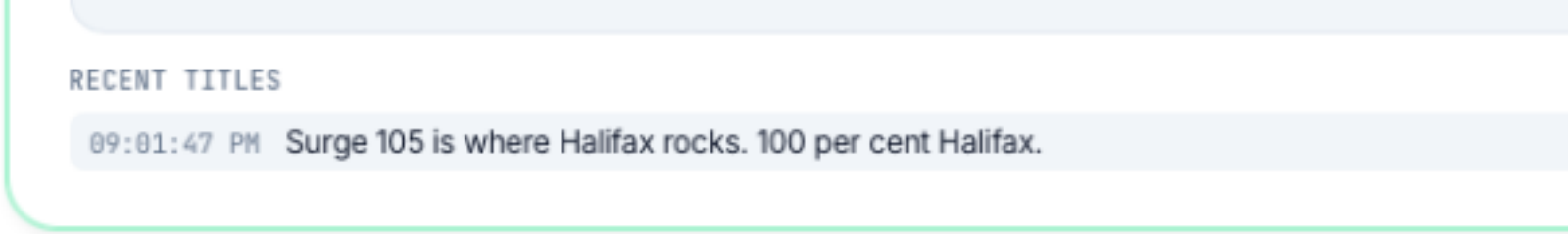
Prerequisites

Docker & Docker Compose

(Optional for Off-Air SDR): One or more RTL-SDR USB dongles (RTL2832U, R820T/R820T2/E4000, RTL-SDR v3/v4).

1. Clone & Start

```
# Clone the repository
git clone https://github.com/your-org/radio-sonar.git
```



Station Monitor Detail

⚡ Key Features

📻 Off-Air RTL-SDR Demodulation Engine

Analog FM Stereo + RDS/RBDS: Spawns `rtl_fm` piped directly into `redsea` for real-time decoding of Program Service (`PS`), RadioText (`RT`), and RadioText+ (`RT+` Artist / Title).

NRSC-5 Digital HD Radio: Demodulates digital HD subchannels (`HD1` through `HD4`) with live MER (Modulation Error Ratio dB), BER, and broadcast station telemetry.

Single-Stick Wideband FM Channelizer: Demodulate multiple nearby FM stations concurrently from a single 2.4 MHz IQ spectrum slice on **one USB stick** via high-speed vectorized NumPy/SciPy DSP (`wideband_channelizer.py`).

Smart Auto-Clustering: Automatically detects when stations fit within the 2.4 MHz window, auto-adjusting the RTL center frequency to cover all channels with optimal SNR.

Multi-Dongle Device Pooling: Run 2, 3, 4, or more physical RTL-SDR USB sticks concurrently (`-d 0`, `-d 1`, `-d 2`...).

Built-in EEPROM Serial Burner: Program permanent sequential serial numbers (`00000001`, `00000002`...) into dongle EEPROMs using `rtl_eeprom` to prevent USB port renumbering on reboots.

5-Bar LED Signal Gauge: Live visual signal strength meter with color-coding (Emerald Green, Cyan, Amber, Red) and exact dBFS/MER readouts.

🎯 Ad Detection & Acoustic Fingerprinting

Hybrid Multi-Signal Detection: Network-specific Triton cue triggers, ICY metadata changes, blank title filters, silence/volume drop detectors.

Dual-Fingerprinting: Hybrid Shazam + Chromaprint music detection that cuts CPU usage by up to 70% during song playback.

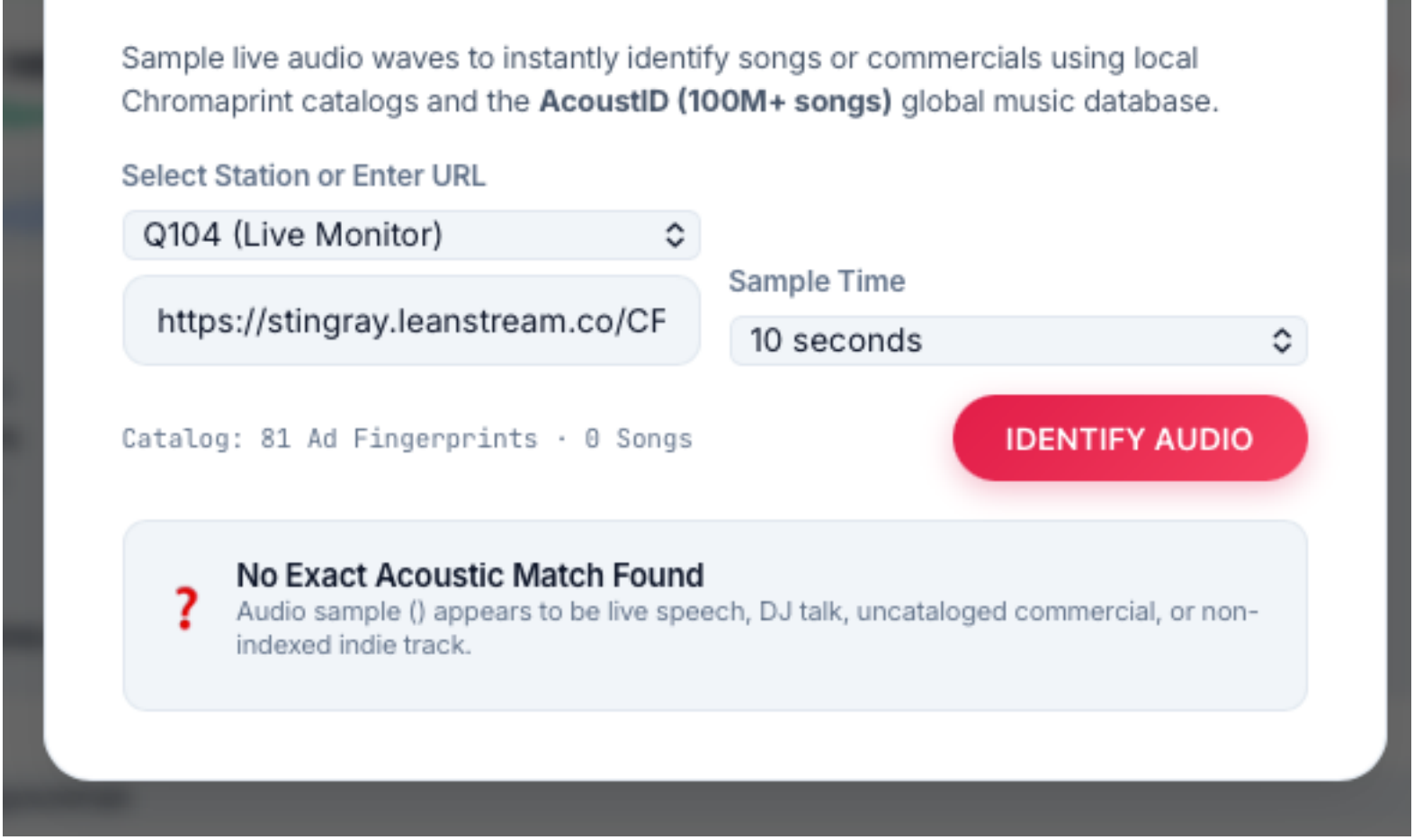
Automatic Break Recording: Segments and archives commercial breaks as MP3 files with customizable retention hours and auto-purge.

🎙️ Automated AI Transcription & Intelligence

OpenAI Whisper AI: Automatically transcribes captured commercial breaks into searchable text with advertiser attribution.

Advertiser Analytics: Track commercial airtime, frequency, and market share across stations and networks.

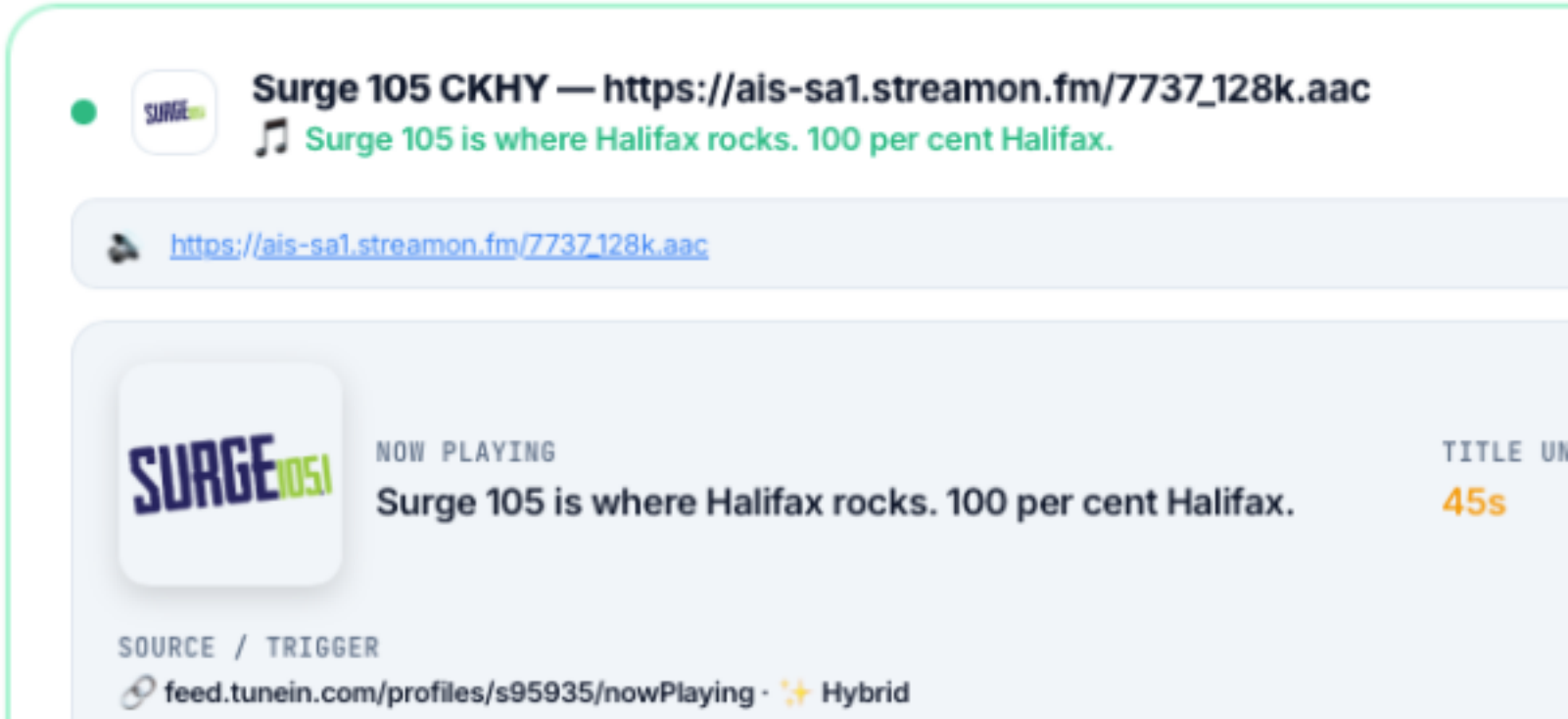
Batch Controls: 1-click `● REC ALL` and `🎙 AI ALL` controls for managing entire station fleets.

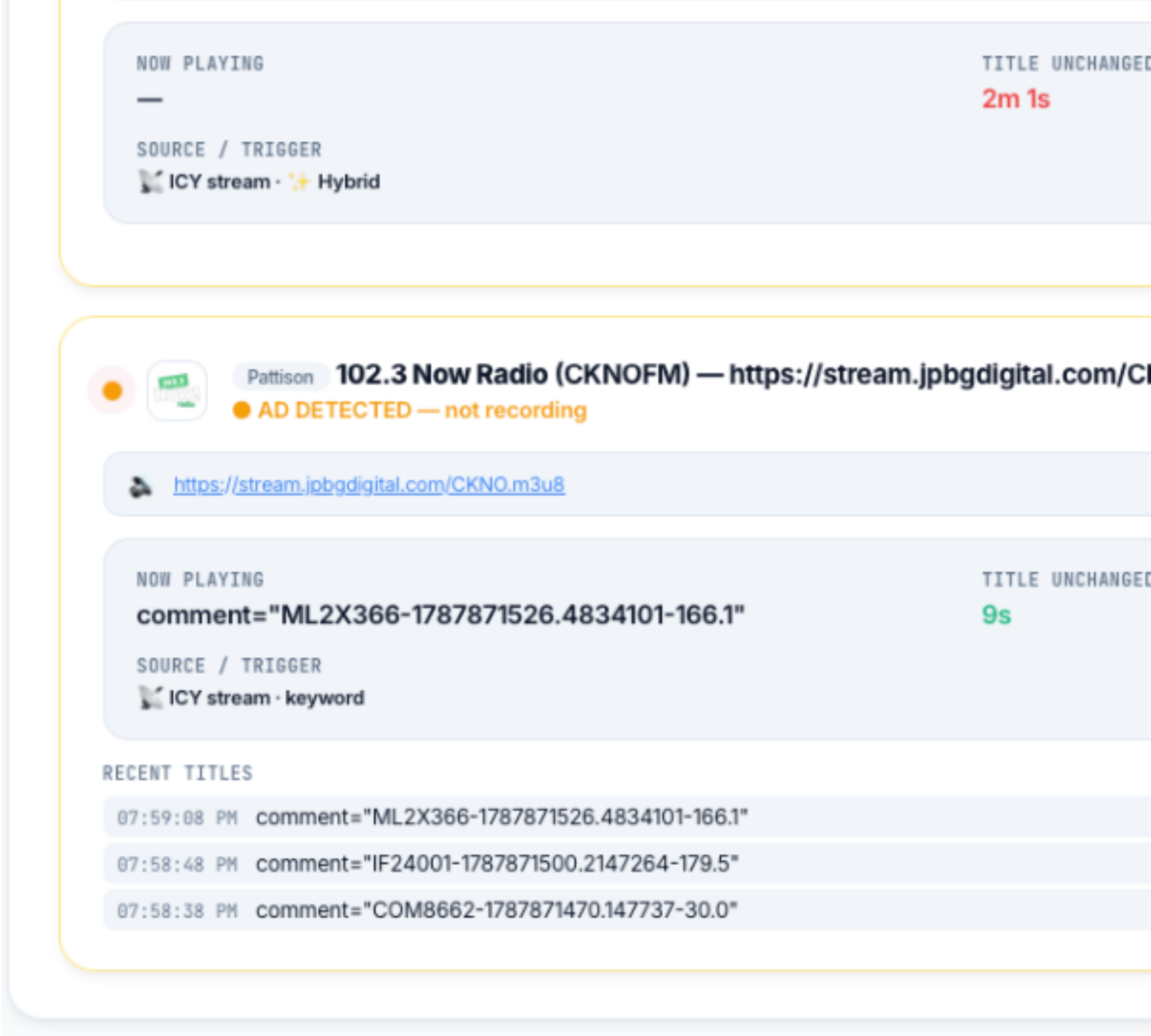


Acoustic Identifier

4. Station Metadata & Cue Tracking

Instant RDS RadioText, now-playing music titles, and time-since-last-change tracking.



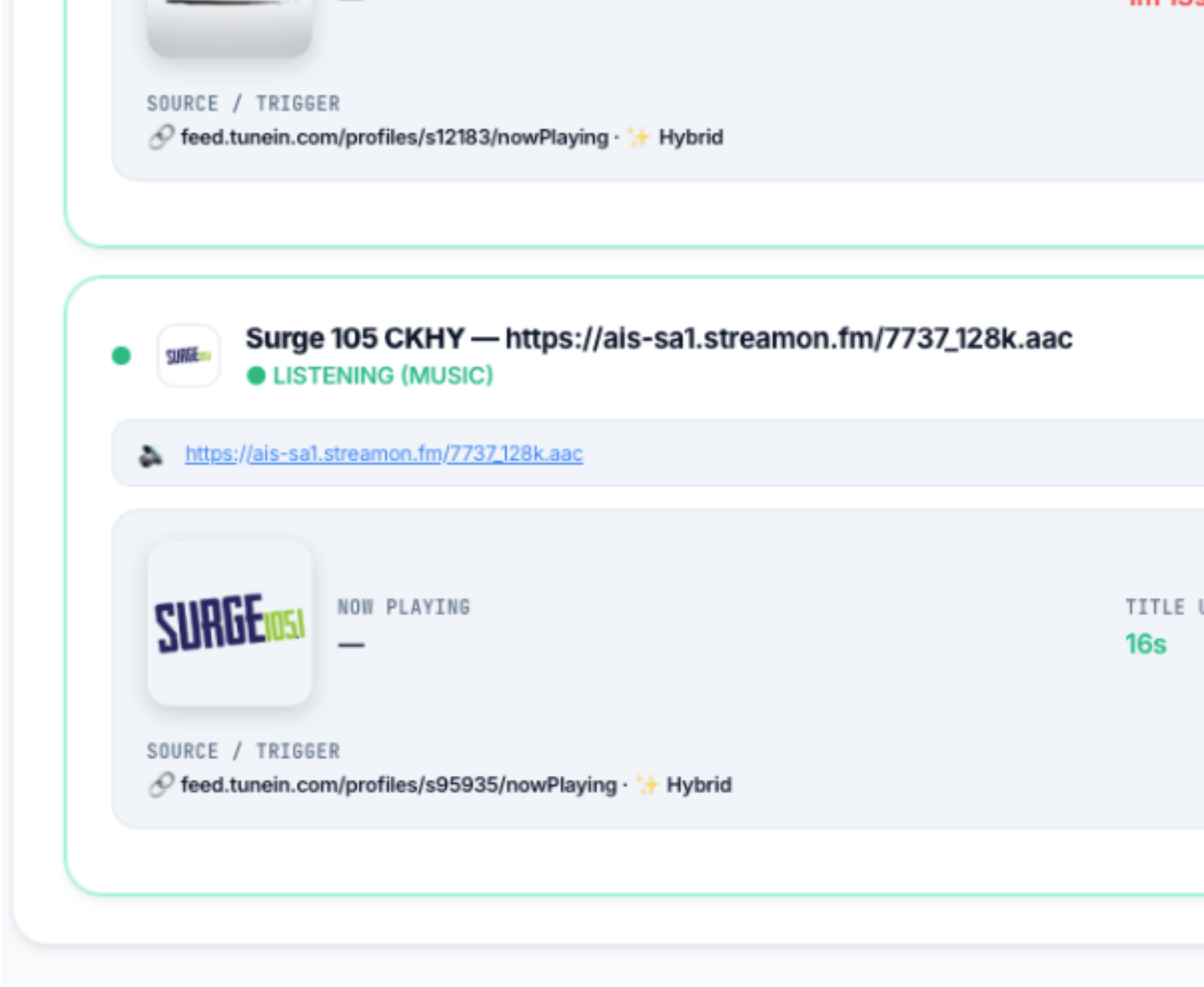


Commercial Detection

3. Acoustic Audio Identifier

Instantly sample live broadcast waves to identify songs or unindexed commercials against Chromaprint catalogs and the AcoustID database.

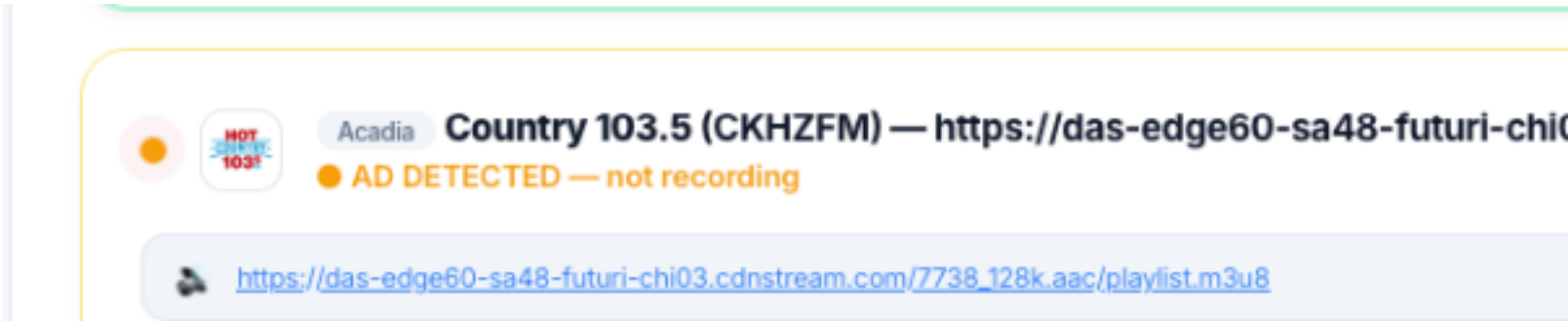


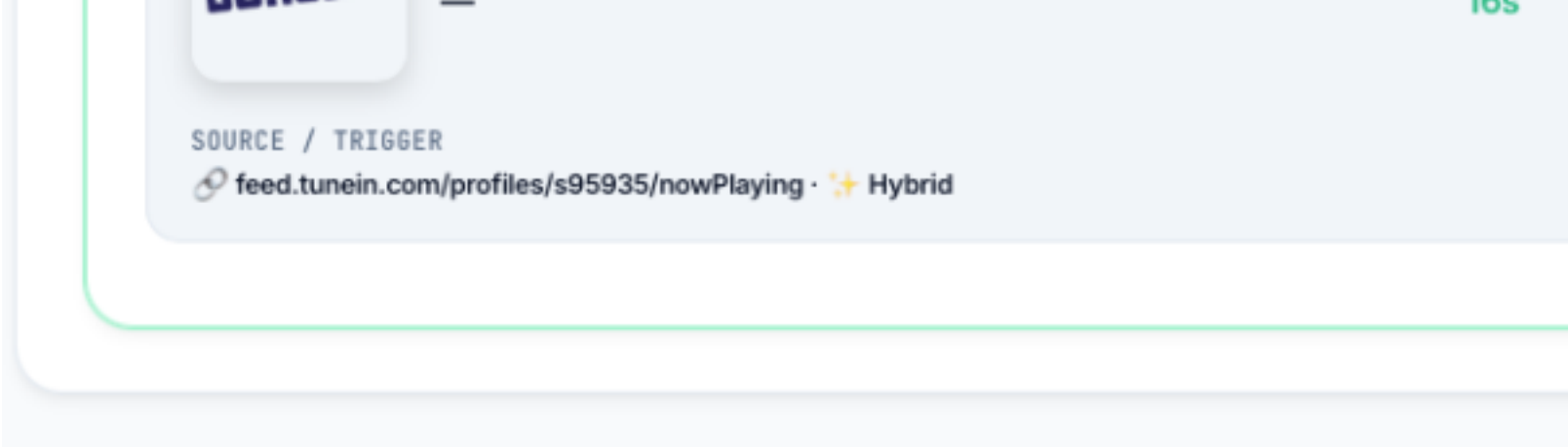


Live Station Monitoring

2. Intelligent Commercial Break & Ad Detection

Multi-signal ad boundary detection using Triton metadata, ICY comment cues, silence/volume drop DSP, and Chromaprint acoustic fingerprints.





Radio SONAR Active Monitors

🌟 Overview

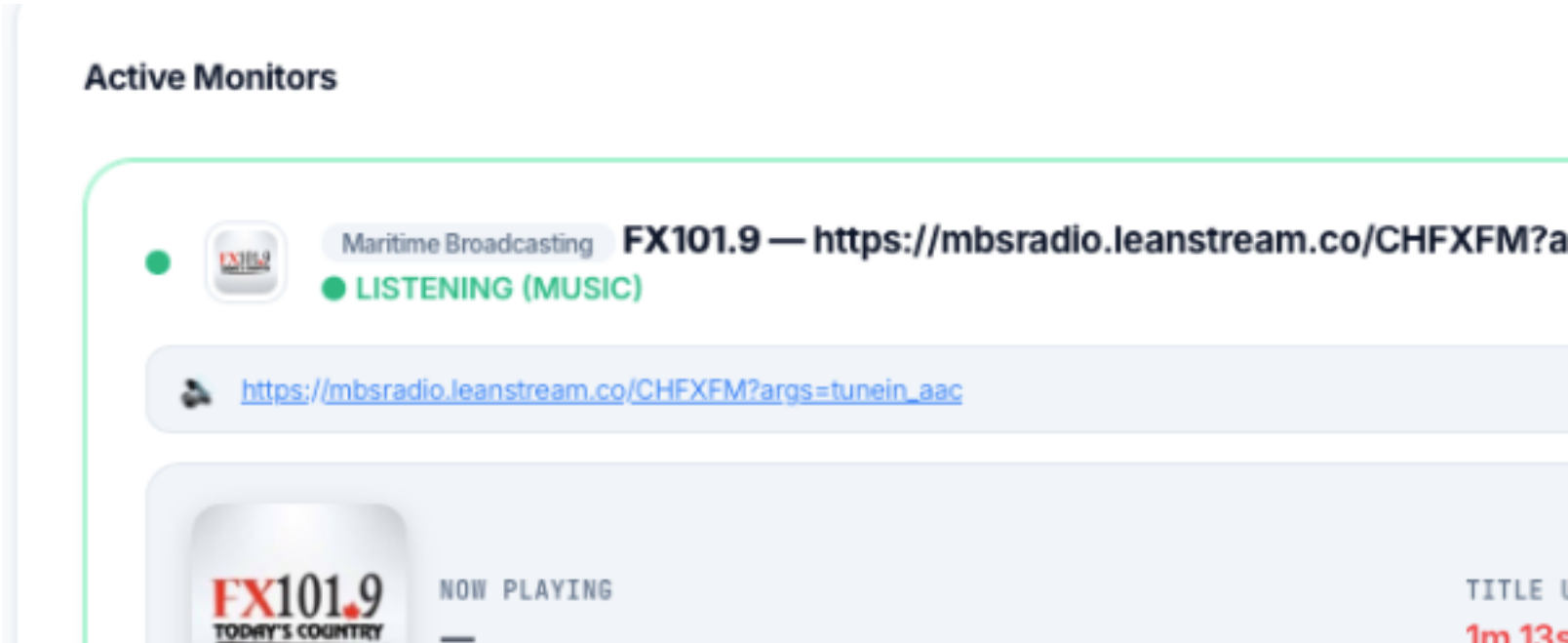
Radio SONAR is a high-performance broadcast monitoring and intelligence platform designed to listen to, analyze, record, and transcribe radio broadcasts in real time.

It seamlessly unifies **Web Audio Streams** (HLS, ICY, AAC, MP3, Triton, Corus, Rogers, TuneIn) and **Off-Air RTL-SDR Receivers** (Analog FM with real-time RDS/RBDS and NRSC-5 Digital HD Radio HD1–HD4) with automated ad break boundary detection, acoustic audio fingerprinting, and local OpenAI Whisper AI transcription.

📸 Visual Tour

1. Live Multi-Station Monitoring & RF Telemetry

Real-time audio player, metadata tracking, album artwork lookup, stall detection, and 5-bar RF signal level meter.



Docker

Node.js

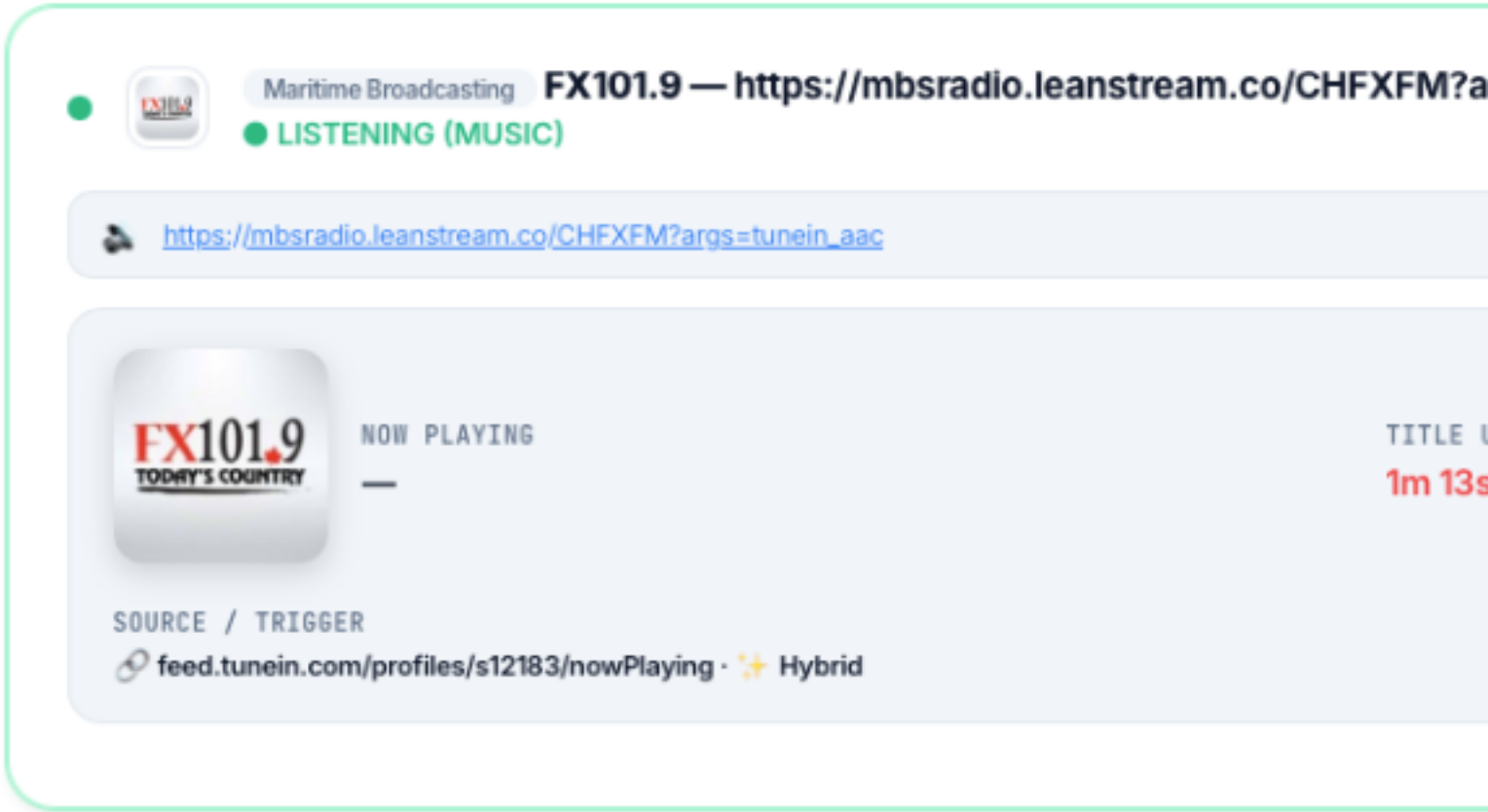
Python

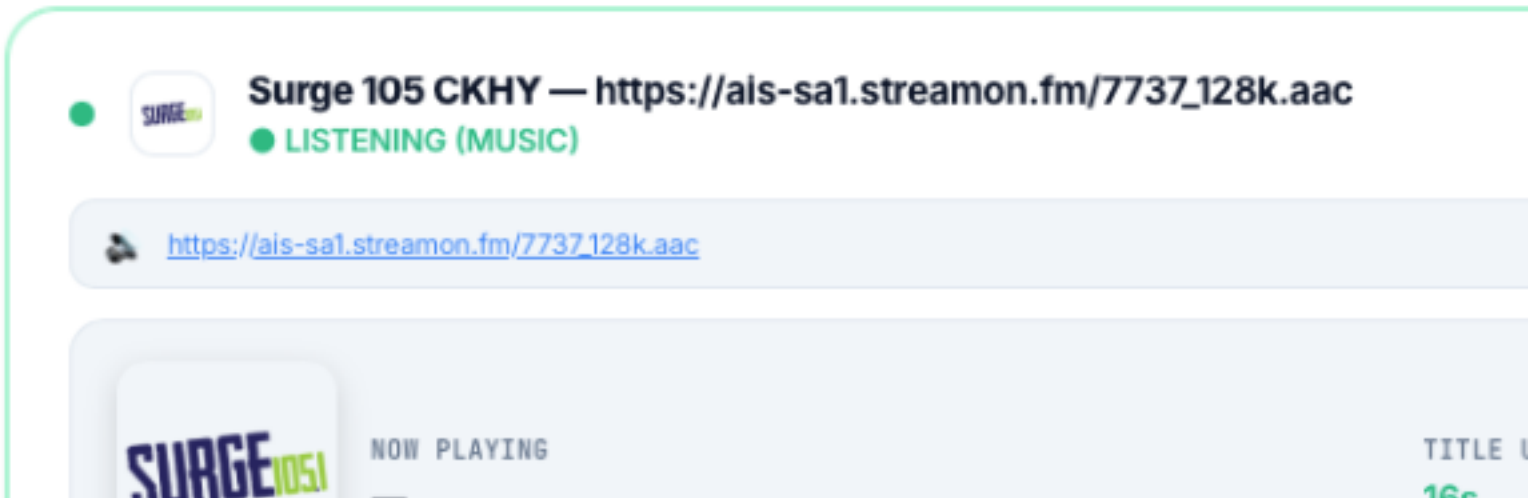
Whisper AI

RTL-SDR

MCP

Active Monitors





Radio SONAR 📻⚡

Real-Time Broadcast Intelligence, Commercial Detection & AI Audio Platform

`Version`

Docker

Node.js

Python

Whisper AI

RTL-SDR

MCP

Active Monitors

